

Linear Algebra with Numpy

Numpy provides a powerful numpy.linalg module for performing linear algebra operations efficiently.

(i) Creating vectors and matrices

import numpy as np

vector

v = np.array([1, 2, 3])

Matrix

A = np.array([[1, 2],
[3, 4]])

B = np.array([[5, 6],
[7, 8]])

(ii) Matrix adding and subtraction

print(A+B)

[[6 8]
[10 12]]

print(A-B)

[[-4 -4]
[-4 -4]]

(iii) Matrix Multiplication

Use the @ operator or np.matmul()

C = A @ B

print(C)

[[19 22]
[43 50]]

(iv) Dot product

x = np.array([1, 2, 3])

y = np.array([4, 5, 6])

print(np.dot(x, y))

32

(v) Transpose

print(A.T)

[[1 3]
[2 4]]

(vi) Determinant

det = np.linalg.det(A)
print(det)

-2.0

(vii) Matrix inverse

A_inv = np.linalg.inv(A)
print(A_inv)

[[-2. 1.]
[1.5 -0.5]]

(viii) Solving linear equation

A = np.array([[3, 1],
[1, 2]])

Solution: $Ax = B$

B = np.array([9, 8])

x = np.linalg.solve(A, B)
print(x)

(ix) Eigen values and Eigen vectors

eigenvalues, eigenvectors = np.linalg.eig(A)

print("Eigenvalues =", eigenvalues)
print("Eigen vector =", eigenvector)

Eigen value $\rightarrow |A - \lambda E| = 0$

Eigen vector $\rightarrow (A - \lambda E)x = 0$

(x) Rank of matrix

A rank of matrix is the maximum number of linear independent rows or columns in the matrix.

```
import numpy as np
A = np.array([[1, 2],
              [2, 4]])
```

```
rank = np.linalg.matrix_rank(A)
print("Rank = ", rank)
```

Rank = 1.

(xi) Checking if matrices are equal

```
np.array_equal(A, B)
```

or

```
np.allclose(A, B)
```

(xii) Linear Combinations

```
u = np.array([2, 1])
v = np.array([-1, 2])
```

$a = 2$
 $b = 3$

```
w = a * u + b * v
print(w)
```

Data Distribution in numpy

* A data distribution in numpy describes how the values of a dataset are spread or distributed.

* In Numpy, the `np.random` module can generate data the follows different statistical distribution.

* A data distribution is the pattern in which data values occur over a range of values.

Common distributions in numpy

(i) Uniform Distributions

A uniform distribution is a probability distribution in which all values within a given interval/range have equal probability of occurring.

Ex: In uniform distribution, every value in range (1,10) has the same chance of being selected.

Types:

* Discrete uniform Distribution: Individual values have equal probability.
Ex: Rolling a dice.

* Continuous Uniform Distribution: Every value within the interval has equal probability density.

Ex:

```
import numpy as np
x = np.random.uniform(1, 10, 5)
print(x)
```

[3.24 8.71 5.12 1.83 6.43]

(i) Normal Distribution:-

It is a probability distribution in which most data values are concentrated around the mean, and fewer values occur as we move away from the mean.

Ex:- Suppose the marks of students are:-

Mean = 60

Standard Deviation = 10.

Most students will have marks around 60, while fewer students will have very low or very high marks.

Ex:- import numpy as np

x = np.random.normal(60, 10, 5)

print(x)

[55.23 68.41 81.72 47.89 71.25]

Syntax: np.random.normal(loc, scale, size)

↓ ↓
Mean standard
 Deviation.

Real life Examples

* Height of People

* Exam marks

* Measurement errors

* B.P measurement

(ii) Binomial Distribution

* The binomial distribution is a probability distribution that tells us the probability of getting a certain number of successes in fixed number of trials.

* Each trial has two possible outcomes → Success (0.5)
→ Failure (0.5)

Ex:- import numpy as np

no. of trials

x = np.random.binomial(5, 0.5, 10)

↑ generate

no. of experiments

print(x)

probability
of success

[3 2 4 1 2 3 2 5 1 3]

Ex:- Tossing a coin 5 times.

(iv) Chi-Square Distribution:-

It is continuous probability distribution mainly used in statistical hypothesis testing. It is especially useful for testing relationship between categorical variables and analyzing variances.

Ex:- Suppose we want to test whether gender and choice of a product are related. we can use a chi-square test of independence.

The chi-square distribution depends on degree of freedom.

Uses:-

* Variance testing

* Statistical analysis

* Goodness of fit test.

* Chi-square test of independence.

Ex!- Suppose we roll a dice 60 times we get:-

1 $\rightarrow$ 8 times

2 $\rightarrow$ 12 times

3 $\rightarrow$ 10 times

4 $\rightarrow$ 9 times

5 $\rightarrow$ 11 times

6 $\rightarrow$ 10 times.

we expect each number:-

$$E = \frac{60}{6} = 10$$

```
import numpy as np
from scipy.stats import chi2square
```

```
observed = np.array([8, 12, 10, 9, 11, 10])
```

```
expected = np.array([10, 10, 10, 10, 10, 10])
```

```
chi, p = chi2square(observed, expected)
```

```
print("Chi-sq. value =", chi)
```

```
print("p-value =", p)
```

```
if p < 0.05:
```

```
    print("Die is not fair")
```

```
else:
```

```
    print("Die is fair")
```

Ex!-

```
import numpy as np
x = np.random.chisquare(df=3, size=5)
print(x)
```

[2.31 4.87 1.25 6.42 3.19]

(v) Poisson Distribution

* It is a probability distribution used to find the probability of a specific no. of events occurring in a fixed interval of time, distance, area or space.

* It is represented by $X \sim \text{Poisson}(\lambda)$.
 $\lambda \rightarrow$ avg. no. of events in given interval.

Uses: * No. of customers arriving at a shop.
* No. of website visitors.
* No. of emails received in an hour.

Ex!- Suppose a call centre receives an average of 4 calls per minute. We can use Poisson distribution to model the no. of calls received in a minute.

```
import numpy as np
```

```
data = np.random.poisson(4, 10)
```

```
print("No. of calls:")
```

```
print(data)
```

```
print("Mean =", np.mean(data))
```

```
print("Variance =", np.var(data))
```

Debugging Python Scripts

RANKA
DATE / /
PAGE

* Debugging means finding and fixing error in a Python program.

* Python provide a built in debugger called pdb and modern IDEs such as vscode and Pycharm provide graphical debugging tools.

(i) Debugging using pdb:-

pdb stands for Python debugger. It allows you to execute a program line by line and inspects variables while the program is running.

Ex:- import pdb

```
def add(a,b):  
    result = a+b
```

pdb.set_trace() → when execution reaches
return result pdb.set_trace(), the program
 pauses and opens the debugger.

x=10

y=20

print(add(x,y))

Simple Flow:-

Run Program → Find Error → Run PDB → Check variables → Step through code
↓
Test again ← Fix Bug ← Find Bug

Common pdb commands

RANKA
DATE / /
PAGE

n → Execute the next line

s → Step into a function

c → Continue Execution

p variable → Print the value of a variable

l → Show nearby source code

q → Quit debugging

h → Show help.

Example:- (pdb) p a // prints a

10

(pdb) p b // prints b

20

(pdb) n // Execute next line

(pdb) p result // prints result

30

(ii) Using breakpoint(): In newer version of python, you can simply use:

```
def calculate(a,b):  
    result = a*b
```

breakpoint() → Automatically starts the debugger
return result at that point.

print(calculate(2,3))

(iii) Debugging using an IDE

* An IDE (Integrated Development Environment) provides a graphical way to debug Python programs.

* Common IDEs: VScode, Pycharm, IDLE

* Steps in VS code

- Open your Python file.
- Click beside a line number to create a breakpoint.
- Start the program using Run and Debug.
- The program stops at the breakpoint.

• Inspect variables in the variables panel.

• Use:-

Continue → run until the next breakpoint

Stepover → execute the current line

Step Into → enter a function.

Step Out → exit a current function

Restart → start debugging again

Stop → stop the program.

Ex:- $a = 10$
 $b = 20$

$c = a + b$

$d = (c * 2)$ → if you put a breakpoint on this the IDE pauses there.

`print(d)`

pdb v/s IDE debugger

pdb	IDE debugger
* Command-line debugger	* Graphical Debugger
* Uses commands such as n.p.c	* Uses buttons.
* Built into python	* Usually provided by the IDE.
* Lightweight	* Easier for beginners.
* Work in terminals.	* Provided variable and call-stack views

Classification of Errors in Python

Python errors can be classified into three types:-

(i) Compile-Time Error:- A compile error is an error detected before a program is executed during the compilation or code checking stage.

Types:-

Syntax Error:- Incorrect language syntax.

Type Error:- Incompatible data types.

Semantic Error:- Invalid meaning according to rules.

Declaration Error:- Incorrect or missing declaration.

Argument Error:- Incorrect no./ type of method argument.

(i) Runtime Errors:- Errors that occur while the program is running.

They are usually called exceptions.

Common runtime errors:-

- ZeroDivisionError
- NameError
- Type Error
- Value Error
- Index Error
- Key Error
- File Not Found Error

(ii) Logical Errors:- The program runs without crashing but gives the wrong output because the logic is incorrect.

Types of Testing commonly used with PDB

(i) Unit Testing:- Tests individual functions or methods.

(ii) Integration Testing:- Tests whether different modules or components work together correctly.

(iii) Functional Testing:- Checks whether the program performs its required functions correctly.

(iv) System Testing:- Tests the complete application as a whole.

(v) Regression Testing:- Checks whether changes or bug fixes have introduced new problems.

(vi) Exception / Error Testing:- Checks whether how the program behaves when errors occur.

Developing unit Test in Python

* Unit Testing is the process of testing individual parts of a program, such as functions or methods, to check whether they work correctly.

* Python provides a built-in module called unittest for developing unit tests.

Steps to develop Unit Tests

(i) Identify the unit to test.

(ii) Create test cases.

(iii) Write the test function (use unittest.TestCase)

(iv) Compare actual and expected results (use methods such as assertEqual)

(v) Run the tests.

Common unit testing Methods

assertEqual(a, b) → checks whether two values are equal.

assertNotEqual(a, b) → checks whether values are different.

assertTrue(x) → checks whether condition is True.

assertFalse(x) → checks whether condition is False.

assertIsNone(x) → checks whether value is None.

assertIn(a, b) → checks whether a exists in b.

Ex:- Suppose we have a function:-

```
def add(a,b):  
    return a+b
```

Create a unit Test:-

```
import unittest
```

```
def add(a,b):  
    return a+b
```

```
class TestAdd(unittest.TestCase):
```

```
    def test_add(self):  
        self.assertEqual(add(10,20), 30)
```

```
    def test_add_negative(self):  
        self.assertEqual(add(-5,-3), -8)
```

```
if __name__ == "__main__":
```

```
    unittest.main()
```

Project skeleton in Python

* A project skeleton is the basic structure or framework of a python project. It organizes the code, tests, documentation and other files so that the project is easy to develop and maintain.

* Makes debugging easier.

* Helps multiple developers work on the project.

File / Folder → Explanation

main.py → Starting point of the application.

src / → Contains the main source code

functions.py → Contains functions / classes used by project.

tests / → Contains test programs

test_functions.py → Test the functions in the project.

requirements.txt → Lists external python packages required.

README.md → Contains project information and instructions.

.gitignore → Specifies files git should ignore.

__init__.py → Makes a folder work as a python package.

Basic Project skeleton

```
my-project /  
├── main.py  
├── requirements.txt  
├── README.md  
├── .gitignore  
├── src /  
│   ├── __init__.py  
│   └── calculator.py  
└── tests /  
    ├── __init__.py  
    └── test_calculator.py
```


JSON File (JavaScript Object Notation)

* JSON is a lightweight format used to store and exchange data. It is commonly used in Python projects, web applications and APIs.

* A JSON file normally has the extension .json.

* Python provides built-in module to read, write and parse JSON data.

Ex:- {
 "name": "Ajay",
 "age": 20,
 "course": "Data science"
}

Loading a JSON file

Loading means loading JSON data from a file into a Python object.

Ex:- import json

with open("student.json", "r") as file:
 data = json.load(file)

print(data)

Output:- {'name': 'Ajay', 'age': 20, 'course': 'Data science'}

* json.load() → reads JSON data from a file and convert it into python dictionary.

Writing to a JSON file

Writing means storing python data in JSON file.

Ex:- import json

```
student = {  
  "name": "Ajay",  
  "age": 20,  
  "course": "DS"  
}
```

with open("student.json", "w") as file:
 json.dump(student, file, indent=4)

* The indent = 4 makes JSON file easier to read

Output:- {
 "name": "Ajay",
 "age": 20,
 "course": "DS"
}

Parsing JSON

It means converting JSON-formatted data into a Python object so that it can be used in program.

Ex:- import json

text = '{"name": "Ajay", "age": 20}'

```
data = json.loads(text)  
print(data["name"])  
print(data["age"])
```

* json.loads() is used when JSON data is available as a string.

Output:- Ajay
20

NOTE:

`json.load()` → Reading JSON from a file

`json.loads()` → Parsing JSON from a string.

`json.dump()` → Writing Python data into a file.

`json.dumps()` → Converting Python data into a JSON string.

JSON conversion

In Python, JSON data can be converted into a dictionary, JSON object or JSON string using json module.

(i) JSON → Dictionary :- Use `json.loads()` when JSON is in string form.

Ex: import json

```
data = {"name": "Ajay", "age": 20}
dictionary = json.loads(data)
```

```
print(type(dictionary)) → <class 'dict'>
print(dictionary) → {'name': 'Ajay', 'age': 20}
```

(ii) Dictionary → JSON :- Use `json.dump()` to store a dictionary in a JSON file.

Ex: import json

```
student = {
    "name": "Ajay",
    "age": 20
}
```

with open("student.json", "w") as file:
 json.dump(student, file, indent=4)

(iii) Dictionary → JSON string :- Use `json.dumps()` to convert dictionary into JSON string.

Ex: import json

```
student = {
    "name": "Ajay",
    "age": 20
}
```

```
json_string = json.dumps(student)
```

```
print(json_string) → {"name": "Ajay", "age": 20}
```

```
print(type(json_string)) → <class 'str'>
```

NOTE: Conversion

function

JSON string → Dictionary

`json.loads()`

Dictionary → JSON file

`json.dump()`

Dictionary → JSON string

`json.dumps()`

JSON file → Dictionary

`json.load()`

JSON schema

JSON schema is a standard used to define and validate the structure of JSON data. It specifies what fields are allowed, what data type they should have and which fields are required.

* ~~Also~~ helps prevent invalid data.

* Useful when working with APIs and configuration files

Important keyword

type → specifies the data type
properties → define the field / attributes.

required → specifies mandatory fields
items → defines the types of elements in an array.

minimum → minimum allowed number

maximum → maximum allowed number

minLength → minimum string length

maxLength → Maximum string length.

JSON data

```
{  
  "name": "Ajay",  
  "age": 20,  
  "course": "Python"  
}
```

JSON schema

```
{  
  "type": "object",  
  "properties": {  
    "name": {  
      "type": "string"  
    },  
    "age": {  
      "type": "integer"  
    },  
    "course": {  
      "type": "string"  
    }  
  },  
  "required": ["name", "age", "course"]  
}
```

Schema validation in Python

* It is a process of checking whether a JSON document follows the rules defined in JSON schema.

* It checks things such as:-

- whether required fields are present
- whether the data types are correct.
- whether values are within allowed limits.
- whether strings have the required length.
- whether array contain the correct types.

Ex:- The jsonschema library can be used
If the data does not follow schema, validate() raises a validation error.

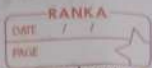
```
import jsonschema import validate
```

```
data = {  
    "name": "Ajay",  
    "age": 20  
}
```

```
schema = {  
    "type": "object",  
    "properties": {  
        "name": {"type": "string"},  
        "age": {"type": "integer", "minimum": 18}  
    },  
    "required": ["name", "age"]  
}
```

```
validate(instance=data, schema=schema)  
print("JSON is valid")
```


Resolving JSON Reference



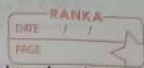
- * JSON reference (`$ref`) is used to reuse a schema or definition.
- * It avoids writing the same schema repeatedly.
- * `#` refers to the current JSON document.
- * The referenced definition is found and used during validation.
- * This makes schema reusable and easier to maintain.

Ex:-

```
{
  "definitions": {
    "student": {
      "type": "object",
      "properties": {
        "name": { "type": "string" }
      }
    }
  },
  "properties": {
    "student": {
      "$ref": "#/definitions/student"
    }
  }
}
```

- * "student" is defined inside ~~definitions~~ definitions.
- * "`$ref`": "`#/definitions/student`" points to that definition.

Extending validator class

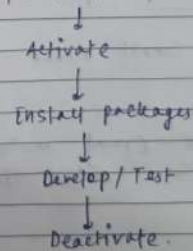


- * It means adding custom validation rules to the existing JSON schema validator.
- * It allows us to create custom keywords for special requirements.
- * Python's `validators.extend()` can be used for this purpose.

Virtual Environments

- * A virtual environment is an isolated environment used to install packages separately for each project.
- * Make projects easier to manage.
- * Prevents package version conflicts.
- * Allows different projects to use different package version.

Basic Process:- Create a Virtual Environment



Floating-Point Arithmetic

* Floating point arithmetic is a method of representing and calculating with real numbers using a fixed number of bits.

* It is commonly used for numbers with fractional parts, such as 3.14, 0.001 or 123.45

Issues:-

- (i) Rounding error:- Result is rounded due to limited precision.
- (ii) Representation error:- some decimal numbers cannot be represented exactly.
- (iii) Overflow:- Number is too large to represent.
- (iv) Underflow:- Number is too small to represent.
- (v) Loss of significance:- Important digits are lost.
- (vi) Accumulation:- Small errors build up over many calculations.
- (vii) Comparison Error:- Equal mathematical values may differ internally.

Regular Expression (Regex)

* A regular Expression (Regex) is a sequence of characters used to search, match, validate and manipulate text according to a specific pattern.

* In python, regular expressions are provided by re module.

* Uses:-

- Searching for specific word.
- Checking e-mail formats.
- Replacing text
- Splitting strings
- Finding special characters/numbers.

* Special symbols:-

\d → digit

\w → word character

+ → one or more occurrences

* → zero or more occurrences

^ → beginning

\$ → end.

\s → white space

? → zero or one occurrence

[A-Z] → uppercase letters

[0-9] → Any digit

Basic RegEx function

(i) re.match() → Checks for a pattern at the beginning of a string.

Ex:- import re

```
text = "Python is easy"
result = re.match("Python", text)
```

```
print(result) → <re.Match object...>
```

(ii) re.search() → Searches for a pattern anywhere in the string.

Ex:- import re

```
text = "I am learning Python"
```

```
result = re.search("Python", text)
print(result.group()) → Python
```

```
print(result) → <re.Match object; span=(11, 20), match='Python'>
```

(iii) re.findall() → Return all occurrences of a pattern.

Ex:- import re

```
text = "Cat dog Cat bird"
```

```
result = re.findall("Cat", text)
```

```
print(result)
```

```
['cat', 'cat']
```

(iv) re.sub() → Replaces matched text with another value.

Ex:- import re

```
text = "I like Java"
```

```
result = re.sub("Java", "Python", text)
print(result) → I like Python
```

(v) re.split() → splits a string according to a pattern.

Ex:- import re

```
text = "apple, banana, orange"
```

```
result = re.split(",", text)
print(result) → ['apple', 'banana', 'orange']
```

Example:- check a Mobile Number

```
import re
```

```
num = "9876453172"
```

```
if re.fullmatch("[0-9]{10}", num):
```

```
    print("Valid number")
```

```
else:
```

```
    print("Invalid number")
```


OOPs concept in Python

* Object-Oriented Programming (OOP) is a programming approach where a program is designed using classes and objects.

* It helps organise large problems into smaller, reusable and manageable parts.

Class:- It is a blueprint or template for creating objects.

It defines - Attributes (data / properties)
Methods (functions / behaviour)

Object:- An object is an instance of a class.
Objects are created using the class name.

Ex:- class Student:

```
def __init__(self, name, age):  
    self.name = name  
    self.age = age
```

→ Constructor.
} Attributes / Instance variables

```
def display(self):  
    print("Name = ", self.name)  
    print("Age = ", self.age)
```

→ Method

Creating objects.

```
s1 = Student("Ajay", 20)
```

Calling Method:

```
s1.display()
```

Attributes:- Attributes are variables that store the property of an object.

Types:-

(i) Instance Attribute:- They belong to a particular object.
* usually written with self.

Ex:- class Mobile:

(ii) Class Attribute:- They are shared by all objects of a class.
* Directly written inside the class.

Ex:- class Book:

```
Category = "Education" // class attribute
```

```
def __init__(self, name, price):  
    self.name = name // Instance attribute  
    self.price = price // Instance attribute.
```

```
b1 = Book("Python", 500)
```

```
b2 = Book("Maths", 490)
```

```
print(b1.name, ":", b1.price)
```

```
print(b2.name, ":", b2.price)
```

* Self keyword → Refers to the current object

* __init__ → Constructor

Method:- A method is a function defined inside a class.
It represents the behaviour of an object.

Develop applications using oop and methods

DATE / /

PAGE

Main steps:-

- (i) Identify the requirements
- (ii) Create classes
- (iii) Define attributes
- (iv) Define methods
- (v) Create objects
- (vi) Use methods through object

Ex:- Calculator Application

```
class Calculator:
```

```
    def add add(self, a, b):  
        return a+b
```

```
    def subtract(self, a, b):  
        return a-b
```

```
    def multiply(self, a, b):  
        return a*b
```

```
c = Calculator()
```

```
print(c.add(10, 5))
```

```
print(c.subtract(7, 3))
```

```
print(c.multiply(13, 2))
```